

TP3 - Création de l'Écran d'Inscription (Register)

Objectifs

- Créer un formulaire d'inscription complet
- Gérer plusieurs champs avec validation
- Utiliser ScrollView pour les longs formulaires
- Comprendre la navigation entre écrans

Étape 1 : Créer le fichier RegisterScreen.js

```
import React, { useState } from 'react';
```

```
import {
```

```
  View,
```

```
  Text,
```

```
  TextInput,
```

```
  TouchableOpacity,
```

```
  StyleSheet,
```

```
  ScrollView,
```

```
  KeyboardAvoidingView,
```

```
  Platform,
```

```
  Switch
```

```
} from 'react-native';
```

```
export default function RegisterScreen() {
```

```
  // États pour tous les champs
```

```
  const [formData, setFormData] = useState({
```

```
    firstName: "",
```

```
    lastName: "",
```

```
    email: "",
```

```
    password: "",
```

```
    confirmPassword: "",
```

```
    phone: "",
```

```
    acceptTerms: false
  });

  // État pour les erreurs
  const [errors, setErrors] = useState({});

  // Fonction pour mettre à jour un champ
  const updateField = (field, value) => {
    setFormData(prev => ({
      ...prev,
      [field]: value
    }));
  };

  // Effacer l'erreur quand l'utilisateur tape
  if (errors[field]) {
    setErrors(prev => ({
      ...prev,
      [field]: ""
    }));
  }
};

// Validation du formulaire
const validateForm = () => {
  const newErrors = {};

  // Validation du prénom
  if (!formData.firstName.trim()) {
    newErrors.firstName = 'Le prénom est requis';
  }
}
```

```
}
```

```
// Validation du nom
```

```
if (!formData.lastName.trim()) {  
    newErrors.lastName = 'Le nom est requis';  
}
```

```
// Validation de l'email
```

```
if (!formData.email.trim()) {  
    newErrors.email = 'L'email est requis';  
} else if (!/^S+@S+\.S+/.test(formData.email)) {  
    newErrors.email = 'Email invalide';  
}
```

```
// Validation du mot de passe
```

```
if (!formData.password) {  
    newErrors.password = 'Le mot de passe est requis';  
} else if (formData.password.length < 8) {  
    newErrors.password = 'Le mot de passe doit contenir au moins 8 caractères';  
} else if (!/(?=.*[0-9])/).test(formData.password)) {  
    newErrors.password = 'Le mot de passe doit contenir au moins un chiffre';  
}
```

```
// Validation de la confirmation
```

```
if (formData.password !== formData.confirmPassword) {  
    newErrors.confirmPassword = 'Les mots de passe ne correspondent pas';  
}
```

```
// Validation du téléphone
```

```
if (formData.phone && !/^[0-9]{10}$/.test(formData.phone.replace(/\s/g, ""))) {  
  newErrors.phone = 'Numéro de téléphone invalide';  
}
```

```
// Validation des conditions
```

```
if (!formData.acceptTerms) {  
  newErrors.acceptTerms = 'Vous devez accepter les conditions';  
}
```

```
setErrors(newErrors);  
return Object.keys(newErrors).length === 0;  
};
```

```
// Fonction d'inscription
```

```
const handleRegister = () => {  
  if (validateForm()) {  
    console.log('Inscription avec:', formData);  
    alert('Inscription réussie !');  
    // Ici, vous feriez normalement un appel API  
  }  
};
```

```
return (  
  <KeyboardAvoidingView  
    behavior={Platform.OS === "ios" ? "padding" : "height"}  
    style={styles.container}  
  >  
    <ScrollView  
      contentContainerStyle={styles.scrollContainer}
```

```
showsVerticalScrollIndicator={false}
```

```
>
```

```
<Text style={styles.title}>Créer un compte</Text>
```

```
{/* Prénom et Nom sur la même ligne */}
```

```
<View style={styles.row}>
```

```
  <View style={styles.halfWidth}>
```

```
    <Text style={styles.label}>Prénom</Text>
```

```
    <TextInput
```

```
      style={[styles.input, errors.firstName && styles.inputError]}
```

```
      placeholder="Jean"
```

```
      value={formData.firstName}
```

```
      onChangeText={(text) => updateField('firstName', text)}
```

```
    />
```

```
    {errors.firstName && (
```

```
      <Text style={styles.errorText}>{errors.firstName}</Text>
```

```
    )}
```

```
</View>
```

```
<View style={styles.halfWidth}>
```

```
  <Text style={styles.label}>Nom</Text>
```

```
  <TextInput
```

```
    style={[styles.input, errors.lastName && styles.inputError]}
```

```
    placeholder="Dupont"
```

```
    value={formData.lastName}
```

```
    onChangeText={(text) => updateField('lastName', text)}
```

```
  />
```

```
  {errors.lastName && (
```

```
    <Text style={styles.errorText}>{errors.lastName}</Text>
```

```

    })
  </View>
</View>

{/* Email */}
<View style={styles.inputContainer}>
  <Text style={styles.label}>Email</Text>
  <TextInput
    style={[styles.input, errors.email && styles.inputError]}
    placeholder="jean.dupont@email.com"
    value={formData.email}
    onChangeText={(text) => updateField('email', text)}
    keyboardType="email-address"
    autoCapitalize="none"
  />
  {errors.email && (
    <Text style={styles.errorText}>{errors.email}</Text>
  )}
</View>

{/* Téléphone */}
<View style={styles.inputContainer}>
  <Text style={styles.label}>Téléphone (optionnel)</Text>
  <TextInput
    style={[styles.input, errors.phone && styles.inputError]}
    placeholder="06 12 34 56 78"
    value={formData.phone}
    onChangeText={(text) => updateField('phone', text)}
    keyboardType="phone-pad"
  />
  {errors.phone && (
    <Text style={styles.errorText}>{errors.phone}</Text>
  )}
</View>

```

```

/>

{errors.phone && (
  <Text style={styles.errorText}>{errors.phone}</Text>
)}
</View>

```

```

{/* Mot de passe */}

<View style={styles.inputContainer}>
  <Text style={styles.label}>Mot de passe</Text>
  <TextInput
    style={[styles.input, errors.password && styles.inputError]}
    placeholder="Minimum 8 caractères"
    value={formData.password}
    onChangeText={(text) => updateField('password', text)}
    secureTextEntry={true}
  />

```

```

{errors.password && (
  <Text style={styles.errorText}>{errors.password}</Text>
)}

```

```

{/* Indicateur de force du mot de passe */}

{formData.password.length > 0 && (
  <View style={styles.passwordStrength}>
    <View style={[
      styles.strengthBar,
      {
        width: `${Math.min(formData.password.length * 10, 100)}%`,
        backgroundColor:
          formData.password.length < 6 ? '#ff3b30' :

```

```

        formData.password.length < 10 ? '#ff9500' : '#34c759'
    }
  }} />
</View>
))
</View>

{/* Confirmer mot de passe */}
<View style={styles.inputContainer}>
  <Text style={styles.label}>Confirmer le mot de passe</Text>
  <TextInput
    style={[styles.input, errors.confirmPassword && styles.inputError]}
    placeholder="Retapez votre mot de passe"
    value={formData.confirmPassword}
    onChangeText={(text) => updateField('confirmPassword', text)}
    secureTextEntry={true}
  />
  {errors.confirmPassword && (
    <Text style={styles.errorText}>{errors.confirmPassword}</Text>
  )}
</View>

{/* Conditions d'utilisation */}
<View style={styles.termsContainer}>
  <Switch
    value={formData.acceptTerms}
    onValueChange={(value) => updateField('acceptTerms', value)}
    trackColor={{ false: '#767577', true: '#007AFF' }}
    thumbColor={formData.acceptTerms ? '#fff' : '#f4f3f4'}
  />
</View>

```



```

    />

    <Text style={styles.termsText}>

      J'accepte les conditions d'utilisation

    </Text>

  </View>

  {errors.acceptTerms && (

    <Text style={styles.errorText}>{errors.acceptTerms}</Text>

  )}

  { /* Bouton d'inscription */}

  <TouchableOpacity

    style={styles.button}

    onPress={handleRegister}

  >

    <Text style={styles.buttonText}>S'inscrire</Text>

  </TouchableOpacity>

  { /* Lien vers connexion */}

  <TouchableOpacity style={styles.linkContainer}>

    <Text style={styles.linkText}>

      Déjà un compte ? Se connecter

    </Text>

  </TouchableOpacity>

</ScrollView>

</KeyboardAvoidingView>

);
}

```

```

const styles = StyleSheet.create({

```

```
container: {
  flex: 1,
  backgroundColor: '#f5f5f5',
},
scrollContainer: {
  flexGrow: 1,
  paddingHorizontal: 20,
  paddingVertical: 40,
},
title: {
  fontSize: 32,
  fontWeight: 'bold',
  textAlign: 'center',
  marginBottom: 30,
  color: '#333',
},
row: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  marginBottom: 10,
},
halfWidth: {
  width: '48%',
},
inputContainer: {
  marginBottom: 20,
},
label: {
  fontSize: 16,
```

```
marginBottom: 8,
color: '#666',
},
input: {
  borderWidth: 1,
  borderColor: '#ddd',
  borderRadius: 8,
  padding: 15,
  fontSize: 16,
  backgroundColor: 'white',
},
inputError: {
  borderColor: '#ff3b30',
},
errorText: {
  color: '#ff3b30',
  fontSize: 14,
  marginTop: 5,
},
passwordStrength: {
  height: 4,
  backgroundColor: '#e0e0e0',
  borderRadius: 2,
  marginTop: 8,
  overflow: 'hidden',
},
strengthBar: {
  height: '100%',
  borderRadius: 2,
```

```
},
termsContainer: {
  flexDirection: 'row',
  alignItems: 'center',
  marginBottom: 20,
},
termsText: {
  marginLeft: 10,
  fontSize: 16,
  color: '#666',
},
button: {
  backgroundColor: '#007AFF',
  borderRadius: 8,
  padding: 15,
  alignItems: 'center',
  marginTop: 10,
},
buttonText: {
  color: 'white',
  fontSize: 18,
  fontWeight: 'bold',
},
linkContainer: {
  marginTop: 20,
  alignItems: 'center',
},
linkText: {
  color: '#007AFF',
```

```
    fontSize: 16,  
  },  
});
```

Étape 2 : Comprendre les nouveaux concepts

ScrollView

Permet de faire défiler le contenu quand il dépasse la taille de l'écran :

```
<ScrollView  
  contentType={styles.scrollContainer}  
  showsVerticalScrollIndicator={false}  
>  
  { /* Contenu */ }  
</ScrollView>
```

État d'objet

Au lieu de plusieurs useState, on utilise un seul objet :

```
const [formData, setFormData] = useState({  
  firstName: "",  
  lastName: "",  
  // ...  
});
```

Spread operator

Pour mettre à jour un champ sans perdre les autres :

```
setFormData(prev => ({  
  ...prev,  
  [field]: value  
}));
```

Switch Component

Pour les cases à cocher style iOS/Android :

```
<Switch  
  value={formData.acceptTerms}
```

```
onValueChange={(value) => updateField('acceptTerms', value)}  
/>
```

Étape 3 : Navigation entre écrans

Installation de React Navigation

```
npm install @react-navigation/native @react-navigation/stack
```

```
npx expo install react-native-screens react-native-safe-area-context react-native-gesture-handler
```

Créer App.js avec navigation

```
import React from 'react';  
  
import { NavigationContainer } from '@react-navigation/native';  
  
import { createStackNavigator } from '@react-navigation/stack';  
  
import LoginScreen from './LoginScreen';  
  
import RegisterScreen from './RegisterScreen';
```

```
const Stack = createStackNavigator();
```

```
export default function App() {  
  return (  
    <NavigationContainer>  
      <Stack.Navigator initialRouteName="Login">  
        <Stack.Screen  
          name="Login"  
          component={LoginScreen}  
          options={{ title: 'Connexion' }} />  
        <Stack.Screen  
          name="Register"  
          component={RegisterScreen}  
          options={{ title: 'Inscription' }} />  
      </Stack.Navigator>  
    </NavigationContainer>  
  );  
}
```

```

    />
  </Stack.Navigator>
</NavigationContainer>
);
}

```

Ajouter la navigation dans LoginScreen

```

// Ajouter le paramètre navigation
export default function LoginScreen({ navigation }) {
  // ...

  // Dans le TouchableOpacity pour s'inscrire
  <TouchableOpacity
    style={styles.linkContainer}
    onPress={() => navigation.navigate('Register')}
  >
    <Text style={styles.linkText}>
      Pas encore de compte ? S'inscrire
    </Text>
  </TouchableOpacity>
}

```

Étape 4 : Améliorations UX

Formatage automatique du téléphone

```

const formatPhone = (text) => {
  // Enlever tous les espaces
  const cleaned = text.replace(/\s/g, "");

  // Ajouter des espaces tous les 2 chiffres
  const match = cleaned.match(/(\d{0,2})(\d{0,2})(\d{0,2})(\d{0,2})(\d{0,2})/);
  const formatted = !match[2] ? match[1] :

```

```
match.slice(1, 6).filter(Boolean).join(' ');
```

```
return formatted;
```

```
};
```

```
// Dans le TextInput téléphone
```

```
onChangeText={(text) => updateField('phone', formatPhone(text))}
```

Focus automatique sur le champ suivant

```
// Créer des refs
```

```
const lastNameRef = useRef(null);
```

```
const emailRef = useRef(null);
```

```
// Sur le TextInput prénom
```

```
returnKeyType="next"
```

```
onSubmitEditing={() => lastNameRef.current?.focus()}
```

```
// Sur le TextInput nom
```

```
ref={lastNameRef}
```

```
returnKeyType="next"
```

```
onSubmitEditing={() => emailRef.current?.focus()}
```

Exercices pratiques

1. Ajouter un champ date de naissance

- Utilisez un DatePicker
- Validez l'âge minimum (18 ans)

2. Ajouter la sélection du pays

- Créez un Picker/Select
- Affichez le drapeau du pays

3. Améliorer la validation du mot de passe

- Vérifiez les majuscules

- Vérifiez les caractères spéciaux
- Affichez les critères en temps réel

4. Ajouter une photo de profil

- Utilisez expo-image-picker
- Affichez un aperçu

Gestion des erreurs serveur

```
const handleRegister = async () => {  
  if (validateForm()) {  
    try {  
      setIsLoading(true);  
      // Simuler un appel API  
      const response = await fetch('https://api.exemple.com/register', {  
        method: 'POST',  
        headers: {  
          'Content-Type': 'application/json',  
        },  
        body: JSON.stringify(formData)  
      });  
  
      if (response.ok) {  
        alert('Inscription réussie !');  
        navigation.navigate('Login');  
      } else {  
        alert('Erreur lors de l\'inscription');  
      }  
    } catch (error) {  
      alert('Erreur de connexion');  
    } finally {  
      setIsLoading(false);  
    }  
  }  
}
```

```
}  
}  
};
```

Bonnes pratiques

Sécurité

- Ne jamais stocker les mots de passe en clair
- Utiliser HTTPS pour les appels API
- Valider côté serveur aussi

Performance

- Utiliser useCallback pour les fonctions
- Éviter les re-renders inutiles
- Optimiser les images

Accessibilité

- Ajouter des labels pour les lecteurs d'écran
- Gérer la navigation au clavier
- Utiliser des couleurs contrastées

Pour aller plus loin

- Authentification avec JWT
- Stockage sécurisé avec SecureStore
- Connexion avec réseaux sociaux
- Vérification par email/SMS